Funciones

Funciones

Una función es un grupo de instrucciones, independiente del programa principal, que se puede reutilizar a lo largo de un programa. Las funciones nos ahorran tener que repetir el grupo de instrucciones, sustituyéndolos por simples llamadas a las funciones.

Funciones sin argumentos

Las funciones se identifican por su nombre. Las funciones se definen mediante la palabra reservada **function** seguida del nombre de la función y unos paréntesis, seguidos de un conjunto de instrucciones escritos entre llaves. La llave que abre el bloque de instrucciones se suele escribir al principio de la línea siguiente, no a continuación del nombre de la función.

```
function nombreDelaFuncion()
{
    bloque_de_sentencias
}
```

Las funciones se llaman en el resto del programa escribiendo el nombre de la función seguido de paréntesis. PHP ejecuta el bloque de instrucciones de la función cada vez que se llama a la función.

```
<?php
// Definición de la función prueba()
function prueba()
{
    print "I Hola esto es una funcion";
    print "\n";
}

print "Programa de prueba.\n";
print "\n";
// Llamadas a la función prueba()
prueba();
prueba();
?>
```

```
Programa de prueba.

Hola esto es una función

Hola esto es una función
```

- Si se llama a una función definida en el mismo fichero, la función puede estar definida antes o después de la llamada.

```php
<?php
// Definición de la función prueba()
function prueba()
{
    print "<p>Hola!</p>\n";
}

// Llamada a la función prueba()
prueba();
?>
```

```
<p>Hola!</p>
```

```php
<?php
// Llamada a la función prueba()
prueba();

// Definición de la función prueba()
function prueba()
{
    print "<p>Hola!</p>\n";
}
?>
```

```
<p>Hola!</p>
```

Los nombres de las funciones siguen las mismas reglas de los identificadores de PHP, es decir, deben comenzar por una letra o un guion bajo (_) y el resto de caracteres pueden ser letras, números o guiones bajos (se pueden utilizar caracteres no ingleses como acentos, eñes, etc), pero los **nombres de funciones no distinguen entre mayúsculas o minúsculas.**

La guía de estilo PEAR para PHP recomienda que los nombres de las funciones sigan el estilo camelCase (es decir, sin espacios ni guiones, con la primera palabra en minúsculas y el resto con la primera letra en mayúsculas).

- Si no se llama a una función en el programa principal, las funciones no tienen ningún efecto en la salida del programa (excepto si contienen errores de sintaxis que impidan la ejecución del programa).

```php
<?php
// Definición de la función prueba()
function prueba()
{
    print "<p>I will not waste chalk.</p>\n";
    print "\n";
    print "<p>I will not skateboard in the halls.</p>\n";
    print "\n";
    print "<p>I will not burp in class.</p>\n";
    print "\n";
}

print "<p>Programa de prueba.</p>\n";
?>
```

```
<p>Programa de prueba.</p>
```

- PHP no distingue ente mayúsculas y minúsculas en el nombre de las funciones.

```php
<?php
// Definición de la función prueba()
function prueba()
{
    print "<p>Función de prueba.</p>\n";
    print "\n";
}

// Llamadas a la función prueba()
prueba();
PRUEBA();
PrUeBa();
?>
```

```
<p>Función de prueba.</p>

<p>Función de prueba.</p>

<p>Función de prueba.</p>
```

- En un programa no se pueden definir dos veces la misma función.

```php
<?php
function prueba()
{
    print "<p>Hola!</p>\n";
}

function Prueba()
{
    print "<p>Adios!</p>\n";
}
?>
```

Fatal error: Cannot redeclare Prueba() (previously declared in ejemplo.php:4)
in **ejemplo.php** on line **7**

Independencia de las funciones respecto al programa principal

Las funciones son independientes del resto del programa. En particular, las variables que aparecen en una función son independientes de las variables del programa principal. En principio, ni la función tiene acceso a las variables del programa principal, ni el programa principal tiene acceso a las variables de la función.

- En el siguiente ejemplo, tanto el programa principal como la función utilizan una variable con el mismo nombre ($a), pero para el programa se trata de dos variables distintas.

```php
<?php
// Definición de la función prueba()
function prueba()
{
    $a = 42;
    print "<p>La variable a es $a.</p>\n";
    print "\n";
}

// Damos un valor a la variable
$a = 100;
print "<p>La variable a es $a.</p>\n";
print "\n";
// Llamamos a la función, que da un valor distinto a una variable con el mismo
nombre
prueba();
// Pero en el programa principal, la variable no ha cambiado su valor
print "<p>La variable a es $a.</p>\n";
?>
```

```
<p>La variable a es 100.</p>

<p>La variable a es 42.</p>

<p>La variable a es 100.</p>
```

- En el siguiente ejemplo, la función genera un aviso ya que se hace referencia a una variable no definida. Esto se debe a que la función no da valor a la variable `$a` y la función no tiene acceso a la variable $a del programa principal. Otro detalle a señalar es que el aviso se genera cuando el programa principal llama a la función, es decir, cuando se ejecuta la función.

```php
<?php
// Definición de la función prueba()
function prueba()
{
    // Intentamos escribir el valor de la variable $a
    // pero como no está definida, se produce un aviso
    print "<p>La variable a es $a.</p>\n";
    print "\n";
}

// Damos un valor a la variable
$a = 100;
print "<p>La variable a es $a.</p>\n";
print "\n";
// Llamamos a la función
prueba();
// Volvemos a escribir la variable
print "<p>La variable a es $a.</p>\n";
?>
```

```
<p>Programa de prueba.</p>

<p>La variable a es 100.</p>

<br />
<b>Warning</b>:  Undefined variable $a in <b>ejemplo.php</b> on line <b>5</b><br />
<p>La variable a es .</p>

<p>La variable a es 100.</p>
```

- En el siguiente ejemplo, es el programa principal el que genera el aviso ya que se hace referencia a una variable no definida. Aunque hemos llamado a la función que da valor a una variable `$a`, el aviso se produce porque el programa no tiene acceso a la variable $a de la función.

```php
<?php
// Definición de la función prueba()
function prueba()
{
    // La variable $a sólo está definida en la función, no fuera de ella
    $a = 42;
    print "<p>La variable a es $a.</p>\n";
    print "\n";
}

// Llamamos a la función
prueba();
// Intentamos escribir el valor de la variable $a
// pero como no está definida, se produce un aviso
print "<p>La variable a es $a.</p>\n";
?>
```

```
<p>La variable a es 42.</p>

<br />
<b>Warning</b>:  Undefined variable $a in <b>ejemplo.php</b> on line <b>13</b><br />
<p>La variable a es .</p>
```

Variables globales

Podemos hacer que una función tenga acceso a las variables del programa principal, indicando en el cuerpo de la función los nombres de las variables precedidas de la palabra reservada global.

- En el siguiente ejemplo, la función puede escribir el valor de la variable definida en el programa.

```php
<?php
function prueba()
{
    // Declaramos la variable $a como global
    global $a;

    print "<p>La variable a es $a.</p>\n";
    print "\n";
}

$a = 100;
prueba();
?>
```

```
<p>La variable a es 100.</p>
```

- El acceso a las variables globales es completo, es decir, las variables se pueden modificar, como muestra el siguiente ejemplo:

```php
<?php
function prueba()
{
    global $a;

    // Modificamos la variable $a en la función
    $a = 50;
}

$a = 100;
prueba();
print "<p>La variable a es $a.</p>\n";
?>
```

```
<p>La variable a es 50.</p>
```

Funciones con argumentos (valores)

Podemos enviar datos a una función incluyendo argumentos en su llamada. En la definición de la función deben indicarse los argumentos que se van a recibir, escribiéndolos entre los paréntesis como variables, separadas por comas. En el programa principal, los datos que se

quieren enviar a la función simplemente se incluyen en la llamada a la función escribiéndolos entre los paréntesis, separados por comas. La función guarda los valores en los argumentos (como variables) en el orden indicado en la definición. Los argumentos se pueden utilizar o no en la función, pero si una función se define con argumentos, la llamada a la función debe incluir todos los argumentos indicados en la definición.

```
function nombreDelaFuncion($argumento_1, $argumento_2, etc ...) {
    bloque_de_sentencias
}
```

- En el ejemplo siguiente, enviamos una cadena a la función para que la incluya en la salida de la función.

```
<?php
function saludo($nombre)
{
    print "<p>Hola, $nombre!</p>\n";
}

saludo("Don Pepito");
?>
```

```
<p>Hola, Don Pepito!</p>
```

- En el ejemplo siguiente, enviamos una cadena y un número a la función.

```
<?php
function saludo($nombre, $veces)
{
    for ($i = 0; $i < $veces; $i++) {
        print "<p>Hola, $nombre!</p>\n";
        print "\n";
    }
}

saludo("Don Pepito", 3);
?>
```

```
<p>Hola, Don Pepito!</p>

<p>Hola, Don Pepito!</p>

<p>Hola, Don Pepito!</p>
```

- El ejemplo siguiente produce un error porque en él se llama sin argumentos a una función que tiene definido un argumento.

```
<?php
// Definición de la función prueba($x) con 1 argumento
function prueba($x)
{
    // Esta función no hace nada
}

// LLamamos a la función sin enviar el argumento
prueba();
?>
```

```
Fatal error:  Uncaught ArgumentCountError: Too few arguments to function prueba(), 0
passed in ejemplo.php on line 9 and exactly 1 expected in ejemplo.php:3
Stack trace:
#0 ejemplo.php(9): prueba()
#1 {main}
  thrown in ejemplo.php on line 3
```

- PHP no produce error si se envía un argumento a una función que no tiene definidos argumentos, pero el argumento enviado no se utiliza para nada.

```
<?php
function saludo()
{
    print "<p>Hola!</p>\n";
}

saludo("Don Pepito");
?>
```

```
<p>Hola</p>
```

Funciones con argumentos: paso por valor

Cuando llamamos a una función escribiendo como argumento una variable del programa principal, lo único que se envía es el valor de la variable. En programación está situación se suele llamar "paso por valor".

- En un paso por valor, la función recibe el valor pero no puede modificar la variable del programa principal, como muestra el siguiente ejemplo.

```php
<?php
function prueba($x)
{
    $x = 200;
    print "<p>La variable es $x.</p>\n";
    print "\n";
}

$a = 100;
print "<p>La variable a es $a.</p>\n";
print "\n";
prueba($a);
print "<p>La variable a es $a.</p>\n";
?>
```

```
<p>La variable a es 100.</p>

<p>La variable es 200.</p>

<p>La variable a es 100.</p>
```

- El nombre de un argumento puede coincidir con el nombre de una variable del programa principal, pero PHP las trata como variables distintas.

```php
<?php
function saludo($nombre)
{
    $nombre = "desconocido";
    print "<p>Hola, $nombre!</p>\n";
    print "\n";
}

$nombre = "Don Pepito";
saludo($nombre);
print "<p>Hola, $nombre!</p>\n";
?>
```

```
<p>Hola, desconocido!</p>

<p>Hola, Don Pepito!</p>
```

Funciones con argumentos: paso por referencia

Se puede pasar una variable por referencia a una función y así hacer que la función pueda modificar la variable. La sintaxis es la siguiente:

```php
<?php
    function foo(&$var)
    {
        $var++;
    }
    $a=5;
    foo($a);
    print($a);
?>`
```

```
6
```

Nota: No existe ningún signo de referencia en una llamada a una función, sólo en la definición de la función. Las definiciones de funciones por sí solas son suficientes para pasar correctamente el argumento por referencia.

Los objetos siempre se pasan por referencia, en realidad esto no es del todo cierto. Desde PHP 5, una variable de tipo objeto ya no contiene el objeto en sí como valor. Únicamente contiene

un identificador del objeto que le permite localizar al objeto real. Cuando se pasa un objeto como parámetro, o se devuelve como retorno, o se asigna a otra variable, las distintas variables no son alias: guardan una copia del identificador, que apunta al mismo objeto.

Funciones que devuelven valores

De la misma manera que el programa principal puede enviar valores a una función, una función puede devolver uno o varios valores al programa principal. La palabra reservada return permite indicar la variable que se devuelve.

- El ejemplo siguiente muestra una función que devuelve la suma de los valores recibidos:

```php
<?php
function suma($arg1, $arg2)
{
    return $arg1 + $arg2;
}

$a = 20;
$b = 30;
$suma = suma($a, $b);
print "<p>$a + $b = $suma</p>\n";
print "\n";
print "<p>$a + $b = " . suma($a, $b) . "</p>\n";
?>
```

```
<p>20 + 30 = 50</p>

<p>20 + 30 = 50</p>
```

- La ejecución de una función se interrumpe cuando se ejecuta un return:

```php
<?php
function suma($arg1, $arg2)
{
    return $arg1 + $arg2;
    // Esta instrucción no se ejecuta nunca.
    print "<p>Hola</p>\n";
}

$a = 20;
$b = 30;
print "<p>$a + $b = " . suma($a, $b) . "</p>\n";
?>
```

```
<p>20 + 30 = 50</p>
```

- Una función puede devolver varios valores, devolviendo una matriz. Desde PHP 7.1 (publicado en diciembre de 2016) para recoger los valores devueltos se pueden utilizar corchetes. Técnicamente, esta operación se denomina desestructurar la matriz y en versiones anteriores se debía hacer con la construcción del lenguaje list(). En el ejemplo siguiente, la función devuelve la suma y el producto de los valores recibidos:

```php
<?php
function sumaProducto($arg1, $arg2)
{
    return [$arg1 + $arg2, $arg1 * $arg2];
}

$a = 20;
$b = 30;
[$suma, $producto] = sumaProducto($a, $b);
print "<p>$a + $b = $suma</p>\n";
print "\n";
print "<p>$a * $b = $producto</p>\n";
?>
```

```
<p>20 + 30 = 50</p>

<p>20 * 30 = 600</p>
```

Funciones con argumentos predeterminados

En PHP se pueden definir funciones con argumentos predeterminados, de manera que si en la llamada a la función no se envía un argumento, la función asume un valor predeterminado. Lógicamente, los argumentos predeterminados deben ser los últimos en la lista de argumentos, para evitar ambigüedades.

Los argumentos predeterminados se establecen en la definición de la función, igualando el nombre del argumento a su valor predeterminado.

El ejemplo siguiente muestra una función que calcula diferentes tipos de media (aritmética, geométrica, armónica). Los argumentos de la función son los números cuya media se debe calcular y el tipo de media a calcular. En el ejemplo, el tipo de media predeterminado es la media aritmética.

```php
<?php
// ESTA ES LA DEFINICIÓN DE LA FUNCIÓN calculaMedia
function calculaMedia($arg1, $arg2, $arg3 = "aritmética")
{
    if ($arg3 == "aritmética") {
        $media = ($arg1 + $arg2) / 2;
    } elseif ($arg3 == "geométrica") {
        $media = sqrt($arg1 * $arg2) / 2;
    } elseif ($arg3 == "armónica") {
        $media = 2 * ($arg1 * $arg2) / ($arg1 + $arg2);
    }
    return $media;
}

// ESTO SON EJEMPLOS DE USO DE LA FUNCIÓN calculaMedia
$dato1 = 12;
$dato2 = 16;

// EL TERCER ARGUMENTO INDICA EL TIPO DE MEDIA A CALCULAR
$media = calculaMedia($dato1, $dato2, "geométrica");
print "<p>La media geométrica de $dato1 y $dato2 es $media.</p>\n";
print "\n";

// AL NO PONER EL TERCER ARGUMENTO, DEVUELVE LA MEDIA ARITMÉTICA
$media = calculaMedia($dato1, $dato2);
print "<p>La media aritmética de $dato1 y $dato2 es $media.</p>\n";
?>
```

```
<p>La media geométrica de 12 y 16 es 6.9282032302755.</p>

<p>La media aritmética de 12 y 16 es 14.</p>
```

Librerías

Las librerías permiten centralizar y reutilizar código, lo que reduce el tamaño y complejidad del código fuente de los programas. El concepto de librerías existen en casi todos los lenguajes de programación, pero en PHP están implementadas de una forma sencilla y fácil de utilizar.

En PHP, una librería es un archivo PHP que se incluye tal cual en otro archivo PHP. Para incluir un archivo, se puede utilizar la expresión include seguida de la ruta hasta el archivo que se quiere incluir, sin necesidad de paréntesis.

Por motivos de seguridad, las librerías deben estar en el mismo servidor que los programas que las incluyen y **el acceso se debe hacer mediante rutas relativas**.

Nota: PHP permitía también acceder a librerías mediante rutas absolutas por http/https. Para ello se necesitaba activar la directiva de configuración allow_url_include, que estaba desactivada por defecto. Pero desde PHP 7.4 (2019) el uso de esta directiva está desaconsejado y si se activa, al intentar cargar una librería remota se genera un error E_DEPRECATED.

Las librerías pueden incluir cualquier tipo de código y pueden ser llamadas en cualquier punto del programa y, en su caso, tantas veces como se quiera. El intérprete de PHP simplemente sustituye las llamadas a las librerías por el contenido de las librerías y ejecuta el resultado obtenido.

En el ejemplo siguiente, un programa incluye dos librerías distintas (una de ellas dos veces). Además las librerías definen y modifican una variable utilizada en el programa. Es importante señalar que al utilizar librerías no se están compartiendo variables entre programas distintos, ya que sólo se está ejecutando un programa, aquel en el que las librerías se incluyen de forma literal. Al ejecutar un programa que incluye librerías, las librerías no se ejecutan de forma independiente, sino únicamente en la medida en que se insertan en el programa ejecutado.

- lib-1.php

```php
<?php
$i = 1;
```

- lib-2.php

```php
<?php
$i = $i + 10;
```

- Programa:

```php
<?php
include "lib-1.php";
print "<p>Ahora \$i vale $i</p>\n";
include "lib-2.php";
print "<p>Ahora \$i vale $i</p>\n";
include "librer-2.php";
print "<p>Ahora \$i vale $i</p>\n";
```

```
<p>Ahora $i vale 1</p>
<p>Ahora $i vale 11</p>
<p>Ahora $i vale 21</p>
```

Las librerías se pueden llamar unas a otras, como en el ejemplo siguiente. Ten en cuenta que la primera instrucción print que se ejecuta es la de lib-2, ya que las librerías se incluyen en el punto en que son llamadas.

- lib-1.php

```php
<?php
include "lib-2.php";
print "<p>Estoy en la librería 1.</p>";
```

- lib-2.php

```php
<?php
print "<p>Estoy en la librería 2.</p>";
```

- Programa:

```php
<?php
include "lib-1.php";
print "<p>Estoy en el programa.</p>";
```

```
<p>Estoy en la librería 2.</p>
<p>Estoy en la librería 1.</p>
<p>Estoy en el programa.</p>
```

Las librerías pueden contener código php, código html o una mezcla de ambos, sin que esto afecte a la forma de incluirlas.

- lib-1.php

```
<p>Estoy en la librería 1.</p>
```

- lib-2.php

```
<?php
print "<p>Estoy en la librería 2.</p>";
```

- lib-3.php

```
<p>Estoy en la librería 3.</p>
<?php
print "<p>Sigo en la librería 3.</p>";
```

- Programa:

```
<?php
include "lib-1.php";
include "lib-2.php";
include "lib-3.php";
print "<p>Estoy en el programa.</p>";
```

```
<p>Estoy en la librería 1.</p>
<p>Estoy en la librería 2.</p>
<p>Estoy en la librería 3.</p>
<p>Sigo en la librería 3.</p>
<p>Estoy en el programa.</p>
```

Cómo se utilizan las librerías

Las librerías o bibliotecas se suelen utilizar para centralizar en un único lugar fragmentos de código que se utilizan en varias páginas. De esa manera, si se quiere hacer alguna modificación, no es necesario hacer el cambio en todos las páginas sino únicamente en la librería.

El problema es que no es fácil que haya fragmentos de código no triviales que se puedan utilizar sin modificaciones en diversos programas. Por ese motivo, las librerías no suelen contener instrucciones sueltas, como en los ejemplos del apartado anterior, sino que suelen contener definiciones de funciones, definiciones de constantes o inicialización de variables.

Sobre este punto, la guía de estilo PSR-1 comenta que en cualquier fichero de una librería se pueden definir funciones, constantes, clases, etc. o se puede incluir parte de la lógica del programa (generar salida, modificar variables, etc.), pero prescribe que no se hagan las dos cosas en un mismo fichero.

Por ejemplo, si definimos en la librería una función que imprima la cabecera de las páginas, desde cualquier página se puede incluir la librería mediante la construcción include y llamar a la función como si se hubiera definido en la propia página:

- lib.php

```php
<?php
function cabecera($titulo)
{
    print "<!DOCTYPE html>\n";
    print "<html lang=\"es\">\n";
    print "<head>\n";
    print "  <meta charset=\"utf-8\">\n";
    print "  <title>$titulo</title>\n";
    print "  <meta name=\"viewport\" content=\"width=device-width, initial-
scale=1.0\">\n";
    print "  <link rel=\"stylesheet\" href=\"estilo.css\" title=\"Color\">\n";
    print "</head>\n";
    print "\n";
    print "<body>\n";
    print "  <h1>$titulo</h1>\n";
    print "\n";
}
```

- pagina-1.php

```php
<?php
include "lib.php";
cabecera("Página de ejemplo");
print "  <p>Esta página es válida</p>";
</body>
</html>
```

```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <title>Página de ejemplo</title>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="estilo.css" title="Color">
</head>

<body>
  <h1>Página de ejemplo</h1>

  <p>Esta página es válida</p>
</body>
</html>
```

- pagina-2.php

```php
<?php
include "lib.php";
cabecera("Otra página de ejemplo");
print "  <p>Esta página también es válida</p>";
</body>
</html>
```

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <title>Otra página de ejemplo</title>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="estilo.css" title="Color">
</head>

<body>
  <h1>Otra página de ejemplo</h1>

  <p>Esta página también es válida</p>
</body>
</html>
```

Potenciales problemas en la carga de librerías

Cuando una librería contiene definiciones de elementos (funciones, constantes, etc.), pueden ocurrir dos problemas:

- Que por algún motivo, no se encuentre la librería (por ejemplo, porque el camino hasta la librería no sea correcto). Las llamadas posteriores a las funciones o constantes provocarían errores en el programa.
- Que por algún motivo, se incluya dos veces a la misma librería (por ejemplo, porque se llame a dos librerías que a su vez llaman a una tercera). Dado que PHP no permite redefinir funciones o constantes con el mismo nombre, la segunda inclusión provocaría errores (E_WARNING en el caso de repetirse la definición de una constante, E_ERROR en el caso de repetirse la definición de una función).

Con relación al primer problema (que no se encuentre la librería), PHP dispone de la expresión require construcciones más, además de include. La diferencia entre ellas es que require produce un error E_COMPILE_ERROR si no se encuentra el archivo (y se interrumpe el programa), mientras que include sólo produce un aviso E_WARNING (y se procesa el resto de la página).

- lib-1.php

```
<?php
print "<p>Estoy en la biblioteca.</p>\n";
```

- Programa:

```php
<?php
include "biblioteca-11.php";
print "<p>Estoy en el programa.</p>";
```

```
Warning:  include(biblioteca-11.php): Failed to open stream: No such file or
directory in ejemplo.php on line 2
Warning:  include(): Failed opening 'biblioteca-11.php' for inclusion
(include_path='...') in ejemplo.php on line 2
Estoy en el programa.
```

- lib-1.php

```php
<?php
print "<p>Estoy en la biblioteca.</p>\n";
```

- Programa:

```php
<?php
require "biblioteca-11.php";
print "<p>Estoy en el programa.</p>";
```

```
Warning:  require(biblioteca-11.php): Failed to open stream: No such file or
directory in ejemplo.php on line 3
Fatal error:  Uncaught Error: Failed opening required 'biblioteca-11.php'
(include_path='...') in ejemplo.php:3
Stack trace: #0 {main} thrown in ejemplo.php on line 3
```

Con relación al segundo problema (que una misma librería se incluya dos veces), PHP dispone de las expresiones include_once y require_once. Ambas expresiones no incluyen las librerías si ya se han incluido anteriormente. La diferencia entre ellas es la misma que
entre include y require, es decir, el error que generan si no encuentran la librería a incluir.

- lib-1.php

```php
<?php
function saludo() {
    print "<p>Hola</p>\n";
}
```

- Programa:

```php
<?php
include "lib-1.php";
include "lib-1.php";
saludo();
```

```
Fatal error:  Cannot redeclare saludo() (previously declared in biblioteca-1.php:4)
in biblioteca-1.php on line 3
```

- lib-1.php

```php
<?php
function saludo() {
    print "<p>Hola</p>\n";
}
```

- Programa:

```php
<?php
include_once "lib-1.php";
include_once "lib-1.php";
saludo();
```

```
<p>Hola</p>
```

La elección de una expresión u otra depende del contenido de la librería y de la estructura de nuestro aplicación:

- Si la librería contiene código ejecutable (no únicamente definiciones) y puede ser llamada varias veces en el mismo programa, lógicamente no debe utilizarse ni include_once ni require_once.

- Si el contenido de las librerías es imprescindible para el buen funcionamiento de la aplicación, como suele ocurrir en la mayoría de los casos, es preferible usar require o require_once a include o include_once, para evitar que se ejecuten de forma incompleta los programas.

La guía de estilo del proyecto PEAR prescribe el uso de la construcción require_once en el caso de librerías cuya inclusión no dependa de ninguna condición y que no se utilicen paréntesis alrededor de los nombres de las bibliotecas.

Sesiones

Una de las limitaciones de las páginas web es que cada página web es un documento independiente. Eso hace que dos programas PHP no puedan, en principio, compartir información.

En principio, la única manera de enviar información de una página a otra es a través de un formulario y se trata de la información que ha introducido el usuario en ese formulario. Como mucho podemos enviar información adicional mediante controles ocultos, pero esto no puede considerarse compartir información, puesto que la página que recibe la información no puede saber si la información ha sido manipulada por el camino, ni quién se la envía. Por compartir información nos estamos refiriendo a utilizar variables comunes en programas distintos, sin que la información salga del servidor.

PHP puede superar esta limitación mediante las sesiones. Las sesiones permiten que páginas distintas puedan acceder a una variable común, la variable $_SESSION.

El ejemplo siguiente muestra dos páginas: en la primera se crea la variable en la variable global $_SESSION, que se puede utilizar en esa misma página. La segunda página, que se encuentra en el mismo servidor, tiene acceso a la variable creada en el primer programa (si se ha ejecutado antes el primer programa, obviamente).

```php
<?php
session_start();

$_SESSION["nombre"] = "Pepito Conejo";
print "<p>El nombre es $_SESSION[nombre]</p>";
```

El nombre es Pepito Conejo

```
//Otra p
<?php
session_start();

print "<p>El nombre es $_SESSION[nombre]</p>";
```

El nombre es Pepito Conejo

Las sesiones no deben confundirse con las cookies. Las cookies es un método que permite guardar información en el ordenador del cliente para recuperarla en el futuro, mientras que en las sesiones la información se mantiene en el servidor hasta que se cierra la sesión (por intervención del usuario o por tiempo).

El trabajo con sesiones tiene tres partes:

- Creación o apertura de la sesión

  Cuando alguna página crea una sesión utilizando la función correspondiente, el servidor asocia al navegador del usuario un identificador de usuario único. El identificador se guarda en el usuario en forma de cookie o, si el navegador del usuario no permite la creación de cookies, añadiendo el identificador en la dirección de la página.

- Utilización de la sesión

  Si ya se ha creado la sesión, las páginas solicitadas por el mismo navegador pueden guardar y recuperar información en el servidor, información que se asocia al identificador de usuario, por lo que no es accesible a otros usuarios. La información se conserva hasta que el usuario o el servidor destruyan la sesión.

- Destrucción o cierre de la sesión

  Tanto el usuario como el servidor pueden cerrar la sesión. El usuario puede destruir la sesión cerrando el navegador. El servidor puede destruir la sesión cuando alguna página utilice la función correspondiente o al cabo de un tiempo determinado.

Crear la sesión / Unirse a la sesión

En PHP, las sesiones se crean mediante la función session_start(). Si la sesión no existía, esta función crea la sesión y le asocia un identificador de sesión único. Si la sesión ya existía, esta función permite que la página se una a la sesión ya existente y tenga acceso a la información

vinculada a la sesión. Es decir, todas las páginas que quieran guardar datos en $_SESSION o leer datos deben comenzar con la función session_start().

```php
<?php
session_start();

$_SESSION["nombre"] = "Pepito Conejo";
print "<p>El nombre es $_SESSION[nombre]</p>";
```

El nombre es Pepito Conejo

```php
<?php
session_start();

print "<p>El nombre es $_SESSION[nombre]</p>";
```

El nombre es Pepito Conejo

La creación de sesiones requiere el envío de cabeceras HTTP, por lo que la función **session_start() debe utilizarse antes de empezar a escribir el contenido de la página**. En caso contrario PHP producirá un aviso y no se creará la sesión. El motivo es que el identificador de sesión se utiliza en las cabeceras de respuesta HTTP y las cabeceras se envían antes del texto de la página. El ejemplo siguiente es incorrecto, ya que utiliza la función session_start() después de haber escrito texto.

```php
<?php
// Este código es incorrecto, la sesión se crea después de crear texto
print "<p>Hola</p>\n";
session_start();
```

Hola

Warning: session_start(): Cannot send session cache limiter - headers already sent (output started at ejemplo.php:3) in **ejemplo.php** on line **4**

Si en una página se repite la llamada a session_start(), se genera un aviso.

```php
<?php
session_start();
session_start();
```

Notice: session_start(): Ignoring session_start() because a session is already active in **ejemplo.php** on line **3**

Utilizar la sesión

Cuando una página ha creado una sesión o ha accedido a una sesión ya existente mediante session_start(), la página tiene acceso a la matriz $_SESSION que contiene las variables de esa sesión.

La variable $_SESSION es un array asociativo en el que se pueden definir valores como en cualquier array asociativo. La diferencia es que es accesible desde cualquier página siempre que tengan la misma sesión.

El ejemplo siguiente muestra dos páginas. La primera página guarda información en $_SESSION y la segunda la utiliza.

```php
<?php
session_start();

$_SESSION["nombre"] = "Pepito Conejo";
print "<p>Se ha guardado su nombre.</p>\n";
```

```
<p>Se ha guardado su nombre.</p>
```

```php
<?php
session_start();

print "<p>Su nombre es $_SESSION[nombre].</p>\n";
```

```
<p>Su nombre es Pepito Conejo.</p>
```

Los nombres de los primeros índices de la matriz $*SESSION tienen que cumplir las mismas reglas que los nombres de las variables, es decir, que el primer carácter debe ser una letra o un*

guion bajo (). En particular, no deben ser números ni contener caracteres no alfanuméricos.

```php
<?php
session_start();

// Este código es incorrecto, ya que PHP no puede crear un índice numérico
// en $_SESSION. Aunque en la ejecución de este programa $_SESSION contiene
// el elemento con índice numérico, y por eso se muestra en la pantalla, la
// sesión se destruye al finalizar la página y no estará disponible para
// otras páginas.
$_SESSION[0] = "Pepito Conejo";
print "<p>Se ha guardado su nombre.</p>\n";
print "<pre>"; print_r($_SESSION); print "</pre>\n";
```

Se ha guardado su nombre.

Array ([0] => Pepito Conejo)

Warning: Unknown: Skipping numeric key 0 in **Unknown** on line **0**

El mensaje de error se produce al finalizar la página, cuando PHP intenta guardar los datos de sesión.

La variable $_SESSION puede tener más de una dimensión y en ese caso los segundos índices ya no tiene la limitación de no poder ser números.

```php
<?php
session_start();

// Este código es correcto, ya que el primer índice no es numérico
$_SESSION["nombres"][0] = "Pepito Conejo";
print "<p>Se ha guardado su nombre.</p>\n";
print "<pre>"; print_r($_SESSION); print "</pre>\n";
```

Se ha guardado su nombre.

Array ([nombres] => Array ([0] => Pepito Conejo))

Cerrar la sesión

Cerrar una sesión es destruir la variable $_SESSION y el identificador de la sesión.

Las sesiones se pueden cerrar de varias maneras:

- El usuario puede cerrar la sesión simplemente cerrando el navegador (no basta con cerrar las pestañas).
- Un programa puede cerrar la sesión mediante la función session_destroy().
- En general, las cookies tienen una duración establecida en la directiva session.cookie-lifetime y el servidor puede borrar la información cuando ha pasado el tiempo indicado en segundos en la directiva session.gc-maxlifetime, pero la duración de una sesión en particular puede establecerse en el momento de su creación mediante la función session_set_cookie_params() (tiempo que se puede modificar posteriormente).

Cuando se destruye una sesión, el programa que ha destruido la sesión sigue teniendo acceso a los valores de $_SESSION creados antes de la destrucción de la sesión, pero las páginas siguientes no. Si se ejecuta el primero de los ejemplos siguientes y después el segundo, se obtienen los resultados indicados:

```php
<?php
session_start();

$_SESSION["nombre"] = "Pepito Conejo";
session_destroy();

if (isset($_SESSION["nombre"])) {
    print "<p>Su nombre es $_SESSION[nombre].</p>\n";
} else {
    print "<p>No sé su nombre.</p>\n";
}
```

```
<p>Su nombre es Pepito Conejo.</p>
```

```php
<?php
session_start();

if (isset($_SESSION["nombre"])) {
    print "<p>Su nombre es $_SESSION[nombre].</p>\n";
} else {
    print "<p>No sé su nombre.</p>\n";
}
```

```
<p>No sé su nombre.</p>
```

Tras destruir una sesión, una página puede volver a unirse a una sesión.

```php
<?php
session_start();
$_SESSION["nombre"] = "Pepito";
session_destroy();

session_start();
$_SESSION["apellido"] = "Conejo";
header("Location:pagina-2.php");
```

```php
<?php
session_start();

if (isset($_SESSION["nombre"])) {
  print "<p>Su nombre es $_SESSION[nombre].</p>\n";
} else {
  print "<p>No sé su nombre.</p>\n";
}

if (isset($_SESSION["apellido"])) {
    print "<p>Su apellido es $_SESSION[apellido].</p>\n";
} else {
    print "<p>No sé su apellido.</p>\n";
}
```

```
<p>No sé su nombre.</p>

<p>Su apellido es Conejo.</p>
```

Nombre de sesión

En principio, cuando el navegador se conecta a un servidor, la sesión es única, es decir, todas las páginas del mismo dominio compartirán la misma varible $_SESSION.

La función session_name() permite establecer un nombre de sesión específico, de manera que todas las páginas que declaren el mismo nombre de sesión accederán a la misma sesión y las que tengan nombres de sesión distintos accederán a variables $_SESSION diferentes.

En el ejemplo siguiente, los dos primeros programas crean la misma variable `$_SESSION, pero como se ha utilizado la función session_name() con nombres diferentes, realmente lo hacen variables de sesión diferentes.

$_SESSION distintas.

El nombre es Pepito Conejo

```
<?php
session_name("ejemplo2");
session_start();

$_SESSION["nombre"] = "Juan Fulánez";
print "<p>El nombre es $_SESSION[nombre]</p>";
```

El nombre es Juan Fulánez

Los dos programas siguientes acceden después a las matrices $_SESSION, pero cada uno accede a la matriz correspondiente al nombre de sesión.

```
<?php
session_name("ejemplo1");
session_start();

print "<p>El nombre es $_SESSION[nombre]</p>";
```

El nombre es Pepito Conejo

```
<?php
session_name("ejemplo2");
session_start();

print "<p>El nombre es $_SESSION[nombre]</p>";
```

El nombre es Juan Fulánez

El nombre de sesión distingue entre minúsculas y mayúsculas, es decir, dos sesiones con el mismo nombre, pero uno en minúsculas y otro en mayúsculas, son dos sesiones distintas.

El nombre de la sesión no puede contener únicamente números, ni tampoco puede contener los caracteres espacio ( ), punto (.), ampersand (&), más (+), corchete izquierdo ([) ni almohadilla (#).

Borrar elementos de la sesión

Los valores de $_SESSION se borran como en cualquier otra variable mediante la función unset().

```php
<?php
session_start();

$_SESSION["nombre"] = "Pepito Conejo";

print "<p>Su nombre es $_SESSION[nombre].</p>\n";

unset($_SESSION["nombre"]);

if (isset($_SESSION["nombre"])) {
    print "<p>Su nombre es $_SESSION[nombre].</p>\n";
} else {
    print "<p>No sé su nombre.</p>\n";
}
```

```
<p>Su nombre es Pepito Conejo.</p>
<p>No sé su nombre.</p>
```

Para borrar todos los valores de $_SESSION se pueden borrar uno a uno o utilizar la función session_unset(), pero normalmente no se debe utilizar unset():

- unset(`$_SESSION`) impide que el resto de la página escriba o lea valores en $_SESSION, pero la sesión conserva los valores, por lo que otras páginas seguirían viendo esos valores.

```php
<?php
session_start();
$_SESSION["nombre"] = "Pepito";
unset($_SESSION);
$_SESSION["apellidos"] = "Conejo";
header("Location:pagina-2.php");
```

```php
<?php
session_start();
$_SESSION["saludo"] = "Hola";
print "<pre>\n";
print_r ($_SESSION);
print "</pre>\n";
```

```
<pre>
Array
(
    [nombre] => Pepito
    [saludo] => Hola
)
</pre>
```

- la función session_unset() borra todos los valores pero permite que el resto de la página (y otras páginas) escriba o lea valores en $_SESSION.

```php
<?php
session_start();
$_SESSION["nombre"] = "Pepito";
session_unset();
$_SESSION["apellidos"] = "Conejo";
header("Location:pagina-2.php");
```

```php
<?php
session_start();
$_SESSION["saludo"] = "Hola";
print "<pre>\n";
print_r ($_SESSION);
print "</pre>\n";
```

```
<pre>
Array
(
    [apellidos] => Conejo
    [saludo] => Hola
)
</pre>
```

Implementación de las sesiones

Al crear una sesión, el servidor asigna un identificador al usuario. Este identificador se guarda en el ordenador del cliente en forma de *cookie*, mientras que en el servidor se guarda un fichero con los valores de las variables de sesión. Cada vez que el cliente solicita una página al servidor, le envía la *cookie* que contiene el identificador y así el servidor puede recuperar los valores de la variable de sesión para utilizarlos al ejecutar la página.

De esta manera, el ordenador cliente no tiene acceso a los valores almacenados en la sesión.

- La página siguiente crea una variable de sesión

```php
<?php
session_name("prueba");
session_start();

$_SESSION["nombre"] = "Barto";

print "<h1>Prueba de sesión</h1>\n";
```

- Al ejecutar la páginas, en el ordenador cliente se crea una *cookie* con el mismo nombre que la sesión.
- En el servidor se guarda la información de las sesiones en ficheros en directorios cuya ubicación se puede establecer con la directiva de PHP session.save_path o modificar en una página con la función sesion_save_path().

  En ese directorio podemos encontrar ficheros de texto plano que incluyen en su nombre el identificador de la sesión.
- El fichero contiene los nombres y valores de las variables de sesión.

Redirección

En ocasiones, al ejecutar un script PHP, nos interesa que el navegador nos dirija automáticamente a otra página (ya sea otro archivo PHP, HTML, o incluso una URL externa).

En **PHP** la redirección se hace normalmente con la función `header()`. El punto clave es que **no puede haberse enviado salida al navegador antes** (ni siquiera un espacio en blanco), porque las cabeceras HTTP deben ir antes del contenido.

Ejemplo básico: redirigir a otra página del mismo sitio

```php
<?php
// Redirige a otro archivo dentro del mismo proyecto
header("Location: pagina2.php");
exit;
```

- `header("Location: ...")` envía al navegador la cabecera HTTP que le dice: "cárgame esta otra URL".
- `exit;` se usa para asegurarse de que el script no siga ejecutando más código después de la redirección.
- Si no se usa el exit, se seguirá ejecutando el php o html, aunque luego se redigirá a la página indicada.

También se puede usar con una URL externa

```php
<?php
header("Location: https://www.google.com");
exit;
```

Vamos a ver un ejemplo en el que nos daría un error.

```php
<?php
echo "Hola, esto es un texto antes de redirigir<br>";

header("Location: pagina2.php"); // Aquí falla
exit;
```

Al ejecutar ese código, PHP intentará enviar la cabecera `Location: pagina2.php`, pero… ya ha enviado texto (`Hola...`). Entonces lanza un **warning** como este:

`Warning: Cannot modify header information - headers already sent by (output started at /var/www/html/ejemplo.php:2) in /var/www/html/ejemplo.php on line 4`

“Headers already sent” significa literalmente que **ya mandaste contenido al navegador (output)** y, por tanto, las **cabeceras HTTP ya viajaron**. Como las cabeceras deben ir siempre al inicio, PHP no puede añadir después un `Location:`.

Si queremos evitar este problema, podemo utilizar javascript para redireccionar desde el cliente en lugar de desde el servidor.

Por ejemplo:

```
<?php
// Supongamos que ya enviaste algo de HTML o echo antes, entonces header()
fallaría
echo "<h1>Bienvenido</h1>";
// Redirección con JavaScript
echo "<script>window.location.href = 'pagina2.php';</script>";
```

También podemos usar:

```
echo "<script>window.location.replace('pagina2.php');</script>";
```

- `window.location.href` → guarda la URL actual en el historial del navegador.
- `window.location.replace` → no la guarda (más parecido a `header("Location:...")`).

Constantes mágicas

Las constantes mágicas en PHP son constantes predefinidas. Están disponibles para cualquier script en el que se ejecuten y cambian según el lugar donde se utilicen. Todas estas constantes «mágicas» se resuelven en tiempo de compilación, a diferencia de las constantes regulares, que se resuelven en tiempo de ejecución.

Hay nueve constantes mágicas en PHP. Estas constantes especiales no distinguen entre mayúsculas y minúsculas.

`__LINE__`

Devuelve el número de línea actual del archivo. En el **ejemplo** siguiente se muestra cómo se puede utilizar esta constante mágica.

```
<?php
   $x="Hello World";
   echo "$x. The current Line number is " . __LINE__ . ".";
?>
```

Producirá el siguiente **resultado**:

```
Hello World. The current Line number is 5.
```

`__FILE__`

Esta constante mágica devuelve la ruta completa y el nombre de archivo del archivo. Si se usa dentro de una inclusión, se devuelve el nombre del archivo incluido.

```
<?php
   $x="Hello World";
   echo "$x. Current PHP script name is " . __FILE__ . ".";
?>
```

Producirá el siguiente **resultado**:

```
Hello World. Current PHP script name is C:\xampp\htdocs\hello.php.
```

`__DIR__`

Esta constante mágica devuelve el directorio del archivo. Si se usa dentro de una inclusión, se devuelve el directorio del archivo incluido. Esto es equivalente a «dirname(**FILE**)». Este nombre de directorio no tiene una barra diagonal final a menos que sea el directorio raíz. Vea el siguiente **ejemplo**:

```
<?php
   $x="Hello World";
   echo "$x. Directory of the Current PHP script name is " . __DIR__ . ".";
?>
```

Mostrará el siguiente **resultado** en el navegador:

```
Hello World. Directory of the Current PHP script name is C:\xampp\htdocs.
```

`__FUNCTION__`

Esta constante mágica devuelve el nombre de la función en la que se usa la constante, o {closure} para funciones anónimas. En el siguiente **ejemplo** se muestra cómo funciona:

```
<?php
   function hello(){
      $x="Hello World";
      echo "$x. The function name is ". __FUNCTION__ . "";
   }
   hello();
?>
```

Producirá el siguiente **resultado**:

```
Hello World. The function name is hello
```

Si esta constante mágica se usa fuera de la función, entonces dará una salida en blanco.

__CLASS__

Esta constante devuelve el nombre de una clase. El nombre de la clase incluye el espacio de nombres en el que se declaró. Vea el siguiente **ejemplo**:

```
<?php
   class myclass {
      public function __construct() {
         echo "Inside the constructor of ". __CLASS__ . PHP_EOL;
      }
      function getClassName(){
         echo "from an instance method of " . __CLASS__ . "";
      }
   }
   $obj = new myclass;
   $obj->getClassName();
?>
```

Producirá el siguiente **resultado**:

```
Inside the constructor of myclass
from an instance method of myclass
```

__METHOD__

La constante **METHOD** devuelve el nombre del método de clase. En el siguiente **ejemplo** se muestra cómo funciona:

```
<?php
   class myclass {
      public function __construct() {
         echo "Calling " . __METHOD__ . " of " . __CLASS__ ."<br>";
      }
      function mymethod(){
         echo "Calling " . __METHOD__ . " of " . __CLASS__ ."";
      }
   }
   $obj = new myclass;
   $obj->mymethod();
?>
```

Producirá el siguiente **resultado**:

```
Calling myclass::__construct of myclass
Calling myclass::mymethod of myclass
```

`__TRAIT__`

Devuelve el nombre del trait . El nombre del trait incluye el espacio de nombres en el que se declaró. En PHP, los trait son un mecanismo para la reutilización de código. Un trait es similar a una clase, pero solo está destinado a agrupar la funcionalidad de una manera detallada y coherente. No es posible crear una instancia de un trait por sí solo.

Echa un vistazo al siguiente **ejemplo**:

```
<?php
   trait mytrait {
      public function hello() {
         echo "Hello World from " . __TRAIT__ ."";
      }
   }
   class myclass {
      use mytrait;
   }
   $obj = new myclass();
   $obj->hello();
?>
```

Producirá el siguiente **resultado**:

```
Hello World from mytrait
```

`__NAMESPACE__`

Esta constante devuelve el nombre del espacio de nombres actual. En PHP, los espacios de nombres nos permiten usar clases/funciones/constantes del mismo nombre en diferentes contextos sin ningún conflicto, encapsulando así estos elementos. Un espacio de nombres es una agrupación lógica de clases/funciones en función de su relevancia.

El siguiente **ejemplo** muestra cómo se puede utilizar esta constante mágica:

```
<?php
   namespace myspace;
   class myclass {
      public function __construct() {
         echo "Name of the class: " . __CLASS__ . " in " . __NAMESPACE__ . "";
      }
   }
   $class_name = __NAMESPACE__ . '\myclass';
   $a = new $class_name;
?>
```

Producirá el siguiente **resultado**:

```
Name of the class: myspace\myclass in myspace
```

ClassName::class

A diferencia de las otras constantes mágicas, esta constante mágica no comienza ni termina con el guión bajo doble (__). Devuelve el nombre completo de la clase.

El siguiente **ejemplo** muestra cómo se puede utilizar esta constante mágica:

```
<?php
   namespace myspace;
   class myclass {
      public function __construct() {
         echo "Name of the class: " . myclass::class ;
      }
   }
   use myspace;
   $a = new myclass;
?>
```

Producirá el siguiente **resultado**:

```
Name of the class: myspace\myclass
```